计算概论　2025 期中考试真题讲评

课程助教：傅翔宇

2025 期中成绩分布

恰好通过题数	0	1	2	3	4	5	6	7	8	9
人数	10	30	50	50	100	100	100	80	80	40

难度（个人评定）

题号	题目名称	主要考点	难度
T1	计算非负整数的位数	循环、字符串	1
T2	模拟考试	二维桶计数	2
T3	病人排队	排序与比较函数	2
T4	竹竿	观察性质 + 排序	3
T5	数据类型转换	观察性质 + 字符串	3
T6	排列	枚举 + 排序	2
T7	操作	观察性质 + 枚举	2
T8	覆盖墙壁	递推	4
T9	k 进制小数减法	高精度及代码实现	5

建议

难度不一定按照题目顺序，像去年的 T5，我个人认为是明显难于 T6,T7 的。

T1：题目原文

题目 - 计算非负整数的位数

来源：From Whf 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

给定一个可能含有前导 0 的非负整数 a，请计算 a 实际的位数。

数据范围：保证 a 实际可以用 long 类型表示，输入值的总长度不超过 20。

关于输入

一行一个可能含有前导 0 的非负整数 a。

关于输出

一行一个整数，表示 a 实际的位数。

计算非负整数的位数：题目描述、数据范围与输入输出格式

T1：题意

给定一个**可能带前导 0** 的非负整数，问它实际有几位。题面保证这个数可以用 long 表示，输入总长度不超过 20。例如 0001234 是 4 位，而 0 与 0000 都是 1 位。

注意点

本题输入可能有前导 0，所以如果使用字符串来判断位数的话要去掉前导零。
另外额外规定 0 是 1 位数，需要特殊判断。

T1：两种做法

按数值不断除以 10

```cpp
long long x;
cin >> x;
int cnt = 0;
while (x > 0) {
    x /= 10;       // 每除一次去掉一位
    ++cnt;
}
cout << max(cnt, 1) << '\n';
```

除几次就是几位。cin 读入时已经消去掉前导 0，所以不必额外处理；要特判 $x = 0$ 的情况。

按字符串数位数

```cpp
string s;
cin >> s;
int p = 0;
while (p + 1 < (int)s.size()
       && s[p] == '0')
    ++p;              // 跳过前导 0
cout << (int)s.size() - p;
```

条件里写 $p + 1 < \text{size}$，是为了让全 0 的串至少保留最后一位，于是 0000 输出 1 而不是 0。

采用做法一需要注意，虽然题目里写了 a 实际可以用 long 类型表示。但是 C++ 里的 long 很可能是 32 位整数（和 int 一样），从而导致溢出。因此**必须**使用 long long。

T2：题目原文

题目 - 模拟考试

来源：xht 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

北京大学正在进行计算概论期中前的模拟考试。

一共有 n 名学生参加，老师为他们精心准备了 m 套模拟考试题，每名学生必须按顺序打完这 m 套模拟考试题。

然而，每名学生都有各自的时间安排，幸运的是，他们在接下来的 k 天中都恰好有 m 天有空参加模拟考试。

比如，小 X 在接下来的第 2,3,5 天有空参加模拟考试，那么他就必须在第 2 天打第 1 套模拟考试题，第 3 天打第 2 套模拟考试题，第 5 天打第 3 套模拟考试题。

再比如，小 Y 再接下来的第 3,4,5 天有空参加模拟考试，那么他就必须在第 3 天打第 1 套模拟考试题，第 4 天打第 2 套模拟考试题，第 5 天打第 3 套模拟考试题。

注意，每名学生都会对题目进行保密，因此第 1 套模拟考试题可以在第 2 天给小 X 做，之后在第 3 天给小 Y 做。

老师需要为每一个人的每一次模拟考试做准备，为了减小工作量，如果在某一天有多个人打同一套模拟考试题，那么老师只需要在这一天准备一场使用这一套题的模拟考试即可。

你作为助教，老师想请你帮他计算一下，他每天需要准备多少场模拟考试。

数据范围：$1 \leq n, m, k \leq 1000$，$m \leq k$。

T2：输入输出与样例

关于输入

第一行三个整数 n,m,k。

接下来 n 行，每行 m 个整数，第 i 行第 j 列的整数 a(i,j) 表示第 i 个人在接下来的 k 天中第 j 个有空的日子为第 a(i,j) 天。

关于输出

一行 k 个整数，第 i 个整数表示接下来的第 i 天老师需要准备的模拟考试场数。

例子输入

样例 1 输入1：
```
2 3 5
2 3 5
3 4 5
```

样例 2 输入2：
```
6 3 7
2 3 4
2 5 7
3 5 7
1 3 5
5 6 7
1 2 3
```

样例 3 输入3：
```
10 10 20
2 3 4 8 9 11 12 16 17 18
2 3 6 10 12 13 14 15 19 20
1 3 7 10 11 13 14 15 17 19
1 2 4 6 7 9 15 17 19 20
2 3 5 6 9 11 14 16 19 20
1 2 3 8 9 10 11 12 15 19
1 4 6 7 9 12 13 17 18 19
1 7 8 9 10 11 13 15 18 20
1 5 6 7 8 9 13 16 18 19
4 5 7 10 11 13 14 17 18 20
```

T2：题意与样例

第 i 名学生的第 j 套题排在第 $a_{i,j}$ 天。对于每一天，我们需要统计同学们做了哪一套题，并且去重。

天	当天要做的事
1	没有安排，0 场
2	人 1 做第 1 套，1 场
3	人 1 做第 2 套、人 2 做第 1 套，2 场
4	人 2 做第 2 套，1 场
5	两人都做第 3 套，只需 1 场

上表是样例 1（$n = 2, m = 3, k = 5$）的逐天分析，输出为 `0 1 2 1 1`。

T2：桶数组去重

设想对于具体的第 k 天，若存在 $a_{i,j} = k$，那么就能说明第 i 名学生在第 k 天做第 j 套题。
一个最简单的思想就是枚举天数 k，枚举所有 (i, j) 先判断 $a_{i,j}$ 是否等于 k，然后再对满足条件的题目 j 去重，但是发现这样会重复枚举 $a_{i,j}$，无法通过。
正确的做法是开一个二维的桶数组 cnt，第一维是天、第二维是试卷标号，表示第 i 天是否有人做第 j 套题。
对于每个 $a_{i,j}$，将 $cnt_{a_{i,j},j}$ 赋值为 1。由于桶数组中，重复赋值 1 不会对答案产生影响，因此完成去重。
最后答案是对于某一天 k 数出 $cnt_{k,\dots}$ 中 1 的个数。

输入（人 × 题）

	题 1	题 2	题 3
人 1	2	3	5
人 2	3	4	5

格子里是" 第几天做"

$\Longrightarrow$

桶数组 cnt[天][题]

	题 1	题 2	题 3	场数
第 1 天	0	0	0	0
第 2 天	1	0	0	1
第 3 天	1	1	0	2
第 4 天	0	1	0	1
第 5 天	0	0	1	1

复杂度是 $O(nm + km)$。

T2：代码

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1005;
int cnt[MAXN][MAXN];
int n, m, k;
int main() {
    cin >> n >> m >> k;
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j) {
            int x; cin >> x;
            cnt[x][j] = 1;    // 置 1，同时去重
        }
    for (int d = 1; d <= k; ++d) {
        int s = 0;
        for (int j = 1; j <= m; ++j) s += cnt[d][j];
        cout << s << (d == k ? '\n' : ' ');
    }
    return 0;
}
```

输出 k 个数在同一行、空格分隔。

T3：题目原文

题目 · 病人排队

来源：xht 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

将登记的病人按照以下原则排出看病的先后顺序：

1. 老年人（年龄 ≥60 岁）比非老年人优先看病。
2. 老年人按年龄从大到小的顺序看病，年龄相同的按登记的先后顺序排序。
3. 非老年人按登记的先后顺序看病。

数据范围：病人的个数不超过 100。

关于输入

第一行一个正整数 n，表示病人的个数；

接下来 n 行，按照病人登记的先后顺序提供病人的信息，包括：一个长度 ≤10 的字符串表示病人的 ID（每个病人的 ID 各不相同且只含数字和字母），一个整数表示病人的年龄，中间用单个空格隔开。

关于输出

按排好的看病顺序输出病人的 ID，每行一个。

病人排队：三条排队原则与数据范围

T3：题意推导

题目直接给定了排序规则，即：老年人（年龄 ≥ 60）优先；老年人内部按年龄从大到小，同龄则按登记先后；非老年人只看登记先后。
题目只要求输出排好序的 ID。
对于这一类排序题，无论规则怎么复杂，排序的框架都不变，要改的只是比较函数 cmp。
对于这道题，你当然可以使用 std::sort，也可以自己手写任意自己喜欢的排序，比如选择排序等.

```
sort(a+1,a+n+1,cmp);//ok
for (int i = 0; i < n - 1; ++i) {
    int minIdx = i;
    for (int j = i + 1; j < n; ++j)
        if (cmp(a[j],a[minIdx])) minIdx = j;
    if (minIdx != i) swap(a[i], a[minIdx]);
}//ok
```

一般来说，cmp 默认是有两个参数（依次为 x,y）的函数。如果我们希望 x 能够排在 y 前面，那么这个函数就返回 1，否则这个函数就返回 0。

T3：样例

例子输入

```
样例 1 输入：
5
021075 40
004003 15
010158 67
021033 75
102012 30
```

例子输出

```
样例 1 输出：
021033
010158
021075
004003
102012
```

提示信息

数据范围：$1 \leqslant n \leqslant 100$。

病人排队：样例（注意 ID 存在前导 0，必须按字符串处理）

T3：比较函数与”排下标”

如何写 cmp

```
bool cmp(int x, int y) {      // x 应排在 y 前？
    bool oldx = age[x] >= 60;
    bool oldy = age[y] >= 60;
    if (oldx != oldy) return ex;  // 一老一非老
    if (!oldx) return x < y;    // 都非老
    if (age[x] != age[y])
        return age[x] > age[y];  // 都老
    return x < y;             // 同龄
}
```

用下标数组排序

```
cin >> n;
for (int i = 1; i <= n; ++i) {
    cin >> name[i] >> age[i];
    p[i] = i;
}
sort(p + 1, p + n + 1, cmp);
for (int i = 1; i <= n; ++i)
    cout << name[p[i]] << '\n';
```

信息分散在 name[]、age[] 两个数组里，如果我们直接用 sort，很难包括两个数组的信息。这里介绍一种小技巧：令 p[i]=i 表示第 i 个登记的人，我们可以对 p 进行排序，这样排序之后，如果 $p[j] = i$ 那么说明第 i 个登记的人在，我们要在第 j 行输出 $name[p[j]]$。

在 cmp 里用 age[x] 取年龄，而且也顺便解决了先登记先看病的条件。

当然，如果你熟练掌握了类和结构体的内容，这一道题就可以写得非常清晰了。

T4：题目原文

题目 - 竹竿

来源：xht 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

有 n 根竹竿，第 i 根竹竿的长度为 ai，在距离一端的 bi 长度处有一个标记。

现要将所有竹竿捆在一起放入竹筒，要求每根竹竿的标记必须重合（会在此处捆上），但每根竹竿的方向可以调换。

请问竹筒需要有多长，才能确保无论每根竹竿是什么方向，捆起来的竹竿都能完全被放进竹筒中。

数据范围：2≤n≤100000，1≤ai≤1,000,000,000，0≤bi≤ai。

关于输入

第一行，一个正整数 n，表示竹竿的数量。

接下来 n 行，第 i 行包含两个整数 ai,bi，描述第 i 根竹竿。

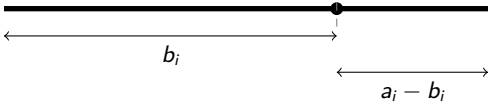
关于输出

一行一个正整数，表示答案。

竹竿：标记点到一端的距离 b_i，方向可调换

T4：题意推导

每根竹竿被标记点分成两段，长度是 b_i 与 $a_i - b_i$；”方向可以调换”就是把这两段左右互换。所有竹竿的标记重合捆在一起，问竹筒至少多长才能保证无论怎么摆都放得下。



提示

第一次看到这类题可能不知道如何下手。发现标记点必须得重复，因此首先不妨固定标记点为原点。这时对于一根竹竿，为了让标记点重合，只能有方向换和不换两种摆法，要对于所有情况都能放下，长度应该取所有摆放方式最长的。
但是对于每根竹竿都枚举是否调换不可接受。
进一步发现比较短的竹竿会被完全覆盖，根本没用。

T4：样例

例子输入

```
样例 1 输入：
2
5 2
4 3

样例 2 输入：
4
5 1
5 2
5 3
5 4
```

例子输出

```
样例 1 输出：
6

样例 2 输出：
8
```

提示信息

数据范围：$2 \leq n \leq 100000$，$1 \leq ai \leq 1,000,000,000$，$0 \leq bi \leq ai$。

竹竿：两组样例，答案分别为 6 与 8

T4：结论

结论

把全部 $2n$ 个数收集成可重集合 $S = \{b_i,\ a_i - b_i\}$，设其中最大的两个数为 $x \geq y$，则答案是

$$\boxed{x + y}.$$

形象地，先把每一根竹竿从标记点砍断，然后再选两根最长的拼在标记两侧。

样例解释

样例 1 的 $S = \{2, 3,\ 3, 1\}$，最大两个是 $3, 3$，答案 6；而两根竹竿本身只有 5 和 4，和却能超过单根长度——这正是"捆在一起"的结果。

T4：结论为什么成立

下界：至少要 $x+y$

若 x 与 y 来自不同竹竿，就把含 x 的那根摆成 x 朝左、含 y 的那根摆成 y 朝右，于是左右各至少要到 x 与 y。若 x,y 是同一根竹竿的两段，那么这根竹竿长为 $x+y$，无论朝哪边，它自己就横跨标记两侧共 $x+y$。

上界：不必超过 $x+y$

任取一种摆放，记左右延伸的长度为 L 与 R。若两者来自不同竹竿，它们是 S 中两个不同的数，和不会超过最大的两个；若来自同一根，则 $L+R$ 是那根竹竿的长，而 x 至少是其中一段、y 至少是另一段，同样不超过 $x+y$。

T4：代码

```cpp
#include <bits/stdc++.h>
using namespace std;
long long v[200005];                // 2n 个数
int main() {
    int n; cin >> n;
    for (int i = 1; i <= n; ++i) {
        long long a, b; cin >> a >> b;
        v[2 * i - 1] = b;           // 标记一侧
        v[2 * i]     = a - b;       // 另一侧
    }
    sort(v + 1, v + 2 * n + 1, greater<long long>());
    cout << v[1] + v[2] << '\n';  // 最大的两个
    return 0;
}
```

复杂度 $O(n \log n)$。两个数之和都可能到 2×10^9 量级，使用 `int` 应该也能通过；

当然你也可以直接使用 `nth_element` 或者扫描一遍求解前两大的数。

T5：题目原文



题目 - 数据类型转换

来源：xht 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

给出一个仅包含 char、bool、int、long long、float、double 六种数据类型的变量 以及 加法(+)运算、乘法(*)运算、逗号(,)运算 三种运算符的表达式 s。请你写程序求出该表达式运算结果的数据类型。

其中，数据类型转换的规则如下：

当进行 + 或 * 运算时：

1. 所有占用字节数小于 int 字节的操作数的类型（如 char）会自动转换为 int。
2. 对两个相同类型的变量 a 和 b，表达式 a+b 和 a*b 的返回值的类型同时与 a 和 b 的类型相同。
3. 对两个整型的运算，会将其转化为 精度更高（占用字节数更多）的数据类型 进行运算。如对于表达式 c+d，若 c 和 d 分别为 int 和 long long 类型，编译器会先将 c 转换为 long long 类型，然后做 long long 类型的加法，运算结果也为 long long 类型。对两个浮点类型的运算，其规则类似。
4. 当整数类型和浮点类型同时参与运算时，所有整数类型都会转换为其中 精度最高的浮点类型。如对于表达式 e*f，若 e 和 f 分别为 long long 和 float 类型，编译器会先将 e 转换为 float 类型，然后做 float 类型的乘法，运算结果也为 float 类型。

当进行逗号(,)运算时：

包含 逗号(,)运算 的表达式 f,g 的返回值为 g，因此，逗号(,)运算 的结果类型与其第二个运算对象相同。注意，逗号(,)运算 的优先级低于 + 与 *。

为了方便，我们只给出表达式中每个变量的类型，而不涉及其变量名称。特别地，long long 用不带空格的 longlong 表示。

数据范围：表达式中数据类型的个数≤100000。

数据类型转换：六种类型与逗号、加、乘三条运算规则

T5：题意推导

表达式里只有六种类型名和三种运算符，要求计算结果**的类型**。规则可以概括成四句话：小于 `int` 的类型参与运算时先提升到 `int`；两个整型取字节数更大的，两个浮点取精度更高的；整型与浮点混合时，结果取其中**最高精度的浮点类型**；逗号表达式取最后一个操作数，且优先级低于加与乘。

先看两个例子

`float*int+longlong` 的结果是 `float`：`int` 与 `longlong` 虽比 `float` 长，但规则规定"浮点胜整型"。`float+bool*double`，`int*longlong+char` 的结果是 `longlong`：逗号只保留最后一段，而这一段里最强的是 `longlong`。

T5：输入输出与样例

关于输入

本题有多组数据，第一行一个正整数 T，表示数据组数。

对于每组数据，一行一个字符串 s，表示给定的表达式。

关于输出

对于每组数据，一行一个字符串，表示给定的表达式的运算结果的类型（类似地，若运算结果的类型为 long long，输出不带空格的 longlong）。

例子输入

样例 1 输入：

```
5
char
int+bool
float*int+longlong
int+char*bool+double
float+bool*double,int*longlong+char
```

例子输出

样例 1 输出：

```
char
int
float
double
longlong
```

提示信息

数据范围：1≤T≤10，表达式中数据类型的个数≤100000。

数据类型转换：输入输出格式与样例）

T5 ： 不会求解一个表达式的值能做这题吗？

题目的要求?

它只要结果**的类型**，不要结果的值。而类型只取决于" 参与了哪些类型"： 加与乘对于类型转换的规则**完全相同**，都是" 取较强"，谁先谁后、加不加括号都不改变结果；

所以不需要栈、递归与优先级表

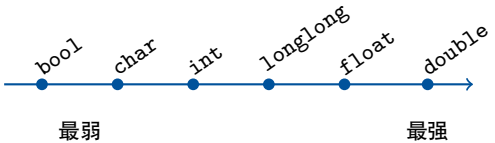
见到加号乘号就想到表达式求值，是一种陷阱；但求值是为了算数，而这里连数的值都不需要，因此这道题没有必要这么麻烦。
况且，作为期中考试的第五道题，根据可以判断这不应该是一道超纲的题。

T5：两个观察与类型强度

观察一：逗号只保留最后一段。先处理最简单的逗号：逗号优先级最低，a，b，c 的值就是 c 的值，所以前面那些段可以整体丢掉，只需要找到最后一个逗号。

观察二：加与乘对结果类型没有区别。两个都是"取较强"，而"取较强"满足交换律与结合律，所以结果就是所有参与类型的最大值。

强弱顺序是 bool、char、int、longlong、float、double。要注意 float 虽然只有 4 字节，却强于 8 字节的 longlong；而 char 与 bool 谁在前都无所谓，它们都会先提升到 int。
但是如果只出现了单个 char 或者 bool 应该直接输出。



轴越靠右越强；参与运算的结果还要截到 int 及更右。

T5：代码

```cpp
string type[6] = {"bool","char","int","longlong","float","double"};
int rk(const string &s) {                  // 类型名 -> 强度
    for (int i = 0; i < 6; ++i) if (s == type[i]) return i;
    return -1;
}
int main() {
    int T; cin >> T; getchar();            // 吃掉第一行末尾的换行
    while (T--) {
        string s; getline(cin, s);         // 表达式里有空格，必须整行读
        int pos = -1;
        for (int i = 0; i < (int)s.size(); ++i)
            if (s[i] == ',') pos = i;      // 最后一个逗号
        s = s.substr(pos + 1);             // 只看最后一段
        int best = -1, cnt = 0; string w;
        for (int i = 0; i <= (int)s.size(); ++i) {
            if (i < (int)s.size() && isalpha(s[i])) w += s[i];
            else if (!w.empty()) { best = max(best, rk(w)); ++cnt; w.clear(); }
        }
        cout << type[cnt == 1 ? best : max(best, 2)] << '\n';
    }
    return 0;
}
```

T6：题目原文

题目 · 排列

来源：xht 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

给出一个 1~n 的排列，询问存在多少个数 i(1≤i≤n-1) 满足前 i 个数和后 n-i 个数排序后都是等差数列。

数据范围：1≤n≤1000。

关于输入

本题有多组数据，第一行一个正整数 T，表示数据组数。

对于每组数据：

第一行一个正整数 n。

第二行 n 个正整数表示排列。

关于输出

对于每组数据：

一行一个整数，表示答案。

排列：切成前后两段，各自排序后都成等差数列

T6：题意推导

给一个 $1 \sim n$ 的排列，问有多少个切分点 i $(1 \le i \le n-1)$，使得前 i 个数排序后是等差数列，后 $n-i$ 个数排序后也是等差数列。判定的对象是"排序后的数列"，与原顺序无关，所以每处都要先排序再检查。

示例

排列 2 1 4 3 6 5 $(n=6)$。切在 $i=2$ 时，前段排序得 $1,2$、后段排序得 $3,4,5,6$，两段都是等差数列；切在 $i=4$ 时前段是 $1,2,3,4$、后段是 $5,6$，也成立。其余切分点都不行：例如 $i=3$ 时前段排序为 $1,2,4$，相邻差是 $1,2$ 并不相同。所以答案是 2 个切分点。

T6：枚举与判定

一共只有 $n-1$ 个切分点，直接枚举即可。对每个切分点，把数组复制一份、两段分别排序，然后检查每段相邻差是否恒定：公差取该段排序后的第一对之差，逐对比较即可。长度不超过 2 的段按定义就成立，不必特判。

```
1 bool check(int l, int r) {
2     if (r - l + 1 <= 2) return true;
3     int d = b[l + 1] - b[l];
4     for (int i = l + 1; i < r; ++i)
5         if (b[i + 1] - b[i] != d) return false;
6     return true;
7 }
```

使用 sort 复杂度是 $O(n^2 \log n)$：n 个切分点，每个切分点排序两段。$n \le 1000$ 时约 10^7，绰绰有余；若在循环里用冒泡排序就退化成 $O(n^3)$，那就可能无法通过了。

T6：代码

```cpp
int n, a[1005], b[1005];
int main() {
    int T; cin >> T;
    while (T--) {
        cin >> n;
        for (int i = 1; i <= n; ++i) cin >> a[i];
        int ans = 0;
        for (int i = 1; i <= n - 1; ++i) {
            for (int j = 1; j <= n; ++j) b[j] = a[j];  // 复制原数组
            sort(b + 1, b + i + 1);                    // 前 i 个
            sort(b + i + 1, b + n + 1);                // 后 n-i 个
            if (check(1, i) && check(i + 1, n)) ++ans;
        }
        cout << ans << '\n';
    }
    return 0;
}
```

每轮都从 a 重新复制，保证判定的是原排列；两个 sort 分别作用在 $[1, i]$ 与 $[i+1, n]$，互不干扰。如果可以使用插入排序，时间复杂度降低到 $O(n^2)$。

当然 n 在 10^6 级别的时候也是可以做的，感兴趣的同学可自行探讨。

T7：题目原文

题目 - 操作

来源：xht 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

有两个正整数 a 和 b，每次可以选择以下操作之一：

1. a←a×2。
2. b←b-1。
3. b←b+1。

求使得 a=b 的最小操作次数。

数据范围：1≤a,b≤1,000,000,000。

关于输入

一行两个正整数 a,b。

关于输出

一行一个非负整数，表示最少操作次数。

操作：三种操作，求使 $a = b$ 的最少次数

T7：题意推导

三种操作分别作用在不同变量上：a 只能乘 2，b 只能加减 1，求让两者相等的最少操作次数。a, b 都不超过 10^9。

终局是什么样

a 的操作比较简单，只能变成 $a \cdot 2^k$。要让 $a' = b'$ ，最终一定有 b' 被调整到某个 $a \cdot 2^k$ 上。因此只要枚举"乘了几次"这一个参数 k，总代价就是乘的次数加上把 b 移到 $a \cdot 2^k$ 的步数。

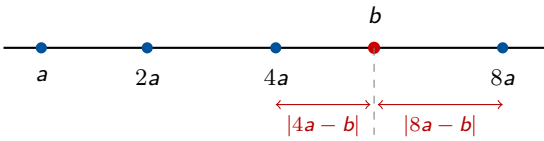
样例

$a = 5$，$b = 8$：乘一次得 10，再把 b 加两次到 10，共 3 步。

T7：如何确定进行几次操作 1

操作顺序无关。乘只改 a、加减只改 b，两者互不影响，所以可以假设"先乘完、再调整 b"。乘 k 次后 a 变成 $a \cdot 2^k$，总代价就是 $k + |a \cdot 2^k - b|$。

一旦超过 b 就会越来越差。再乘一次，代价多了"一次操作"、但距离又翻了一倍，于是单调递增。所以只需枚举到 $a \cdot 2^k$ 刚好跨过 b 的位置；由 $a \geq 1$、$b \leq 10^9$ 可知 k 不超过 30。实现时从 $k = 0$ 开始逐次比较、取最小值即可。



$a \cdot 2^k$ 指数增长，b 只在数轴上一次算一次；跨过 b 之后，右边那段只会更长。

T7：代码

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long a, b;
    cin >> a >> b;
    long long best = abs(a - b);   // k = 0: 一次也不乘
    long long cur  = a;
    for (int k = 1; k <= 30; ++k) {
        cur *= 2;
        best = min(best, (long long)k + abs(cur - b));
        if (cur > b) break;
    }
    cout << best << '\n';
    return 0;
}
```

$a \cdot 2^{30}$ 可能是 10^{18} 量级，但是 b 其实限制了这一点，这里使用 `long long` 只是为了保险。用 `cur` 单独记 $a \cdot 2^k$，不要改动读入的 `a`。

T8：题目原文

题目 - 覆盖墙壁

来源：xht 教师页面 [下一题]

[提交程序] [IDE] [提交记录]

题目描述

你有一个长为 N 宽为 2 的墙壁，给你两种砖头：一种是 L 型覆盖 3 个单元的砖头，另一种长 2 宽 1。如下：



砖头可以旋转，两种砖头可以无限制提供。你的任务是计算用这两种来覆盖 $2\times N$ 的墙壁的覆盖方案数。

例如一个 2×3 的墙可以有 5 种覆盖方法，如下：



注意可以使用两种砖头混合起来覆盖，如 2×4 的墙可以这样覆盖：



数据范围：$1\le N\le 1,000,000$。

由于结果很大，你只需要输出答案对 10000 取模后的值（即不加前导零的后四位数）。为了实现这一点，你可以不断地将结果 %10000。

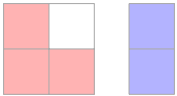
例如可以通过如下方式求出斐波那契数列第 n 项对 10000 取模后的值：

```
f[1] = f[2] = 1;
for (int i = 3; i <= n; i++)
        f[n] = (f[n-1] + f[n-2]) % 10000;
```

覆盖墙壁：L 形（3 格）与 2×1 骨牌（2 格）

T8：题意推导

用两种砖覆盖 $2 \times N$ 的墙：L 形占 2×2 里的 3 格，可以旋转；骨牌占 2 格，横竖都行。求方案数模 10000，N 最大 10^6。

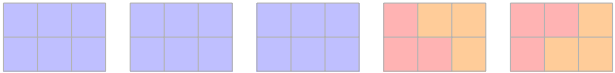


搜索？

搜索每一个块怎么放，当然可以解；但大部分同学应该还没掌握，且不在考试范围内。而且对于这道题，方案数增长很快，$N = 30$ 时就已超过 10^9。因此枚举所有铺法不可行，题目里给的斐波那契数列提示我们必须按列递推——每一列只关心" 前面铺到哪、缺口在哪"。

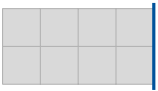
T8：先手算小数据

N	1	2	3	4	5	6	7
方案数	1	2	5	11	24	53	117

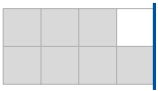


$N = 3$ 的 5 种：三块竖骨牌、一对横骨牌配一块竖骨牌（横对在左或在右），以及两块 L 咬合（互为镜像的两种）。

T8：状态定义



$f[i]$：前 i 列被完整覆盖



$g[i]$：前 i 列中，第 i 列只缺右上角一格

为什么需要第二个状态

只看”前 i 列填满”是不够的：一块 L 横跨两列、只填 3 格，必然在尚未处理的那一列留一个缺口。若只记 f，就丢失了”第 i 列缺一格”这种局面，后面的递推算不下去。而缺右上与缺右下互为镜像，数量相同，所以用一个 g 就可以了，只是需要在递推下一步摆放方案的时候乘 2。

T8：五个转移

递推的思想是一步一步来，先只考虑下一列应该怎么摆放骨牌，并分类讨论。

前两种递推很直接：在第 i 列竖放一块骨牌，前 $i-1$ 列填满就变成前 i 列填满；在第 $i-1$、i 两列各横放一块，就从 $f[i-2]$ 直接到 $f[i]$。

第三种是 L 的形状：它占满第 $i-1$ 列两格、再占第 i 列一格，于是"前 i 列填满、第 i 列缺一格"，进入 $g[i]$；缺上、缺下各一种，各记 1。

反过来，从 $g[i-1]$ 出发，一块 L 正好补上缺口并覆盖第 i 列两格，回到 $f[i]$；或者在缺口处横放一块骨牌，缺口被推到第 i 列的另一侧，仍然留在 g 里。

L 的四种朝向里，有两种会留下空洞（存在一列缺一格），这种空洞再也补不上，所以只计另外两种。



转移 3：L 留下缺口；转移 4：L 补缺口



转移 5：横骨牌把缺口推走

T8：递推式与初值

由转移写出递推

$$f[i] = f[i-1] + f[i-2] + 2\,g[i-1],$$
$$g[i] = g[i-1] + f[i-2].$$

$f[i]$ 里的 $2\,g[i-1]$ 来自"缺右上"与"缺右下"两种镜像局面。初值取 $f[0]=1$、$f[1]=1$、$g[0]=g[1]=0$：空墙算一种方案（否则 $f[2]$ 会算成 1），而 2×1 缺一格只剩一格，任何砖都放不下。

前几项验证

i	1	2	3	4	5	6
$f[i]$	1	2	5	11	24	53
$g[i]$	0	1	2	4	9	20

T8：代码

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MD = 10000;
const int MAXN = 1000005;
int f[MAXN], g[MAXN];          // 必须全局：1e6 个 int 约 4MB
int main() {
    int n; cin >> n;
    f[0] = 1; f[1] = 1;
    g[0] = 0; g[1] = 0;
    for (int i = 2; i <= n; ++i) {
        f[i] = (f[i - 1] + f[i - 2] + 2 * g[i - 1]) % MD;
        g[i] = (g[i - 1] + f[i - 2]) % MD;
    }
    cout << f[n] % MD << '\n';
    return 0;
}
```

每次加法后立刻取模，否则数值会迅速超出 int；$n = 1$ 时循环不执行，直接输出 f[1]=1，正好正确。

T9：题目原文

［提交程序］［IDE］［提交记录］

题目描述

给出 2 个 长的 k 进制数 a 和 b，计算 a-b 的结果值（a 和 b可以带符号）。

如果 k 大于10，数据可以用英文26个大写字母依次表示，如， A, B, C分别表示 10,11,12. ...。设定 k 不大于 36且不小于2。例如16进制，用符号 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

k进制运算的准则是：当被减数对应的位小于减数对应的位时，应向被减数更高的一位借1当成当前位大小为 k 的值。 例如，3进制运算：221-112=102，因为被减数最低位的1比减数最低位的2小，需要借 1，当成最低位的 3（3进制）使用，但被减数最低位已经有 1，借过来后便为4, 然后，4减去减数对应位的 2，得到低位为 2，第2位的被减数被借1之后，变成1了，与减数1相减，结果为0，高位再用2减1为1，最后得到102。加法运算则满足逢 k 进一（对应位的结果大于等于 k时，将当前位结果减去 k，同时向高位进1）。

关于输入

输入 n+1 行，其中第一行 为 n，表示后面有 n 组数；后面的 n 组数格式是：

k a b

其中，三个数之间均用空格间隔。k表示后面两个数的进制，a 和 b均可带符号（也可以不带符号，不带符号则为正数），也可以带小数，如果是纯小数，可以从小数点开始输入（如 .1234），也可以带整数位0（如 0.1234）。假定两个数的长度（连带小数点和符号位）均不超过 100 位。

关于输出

对应的 n 组运算的输出结果。整数部分的最高位不能为0，小数部分的最低位不能为0；如小数部分全部为0，则只输出整数部分；如整数部分为0，则从小数点开始输出；如果运算的结果为0，则输出0.

k 进制小数减法：带符号、带小数，长度不超过 100 位

T9：题意

给两个 k 进制数（$2 \le k \le 36$，10 以上用大写字母表示），算 $a - b$。数可以带符号、可以带小数，长度不超过 100 位。输出要求比较细：整数部分最高位不能是 0，小数部分最低位不能是 0，纯小数以小数点开头，结果为 0 就输出一个 `0`。

输入	输出
10 .12345 123456789.23456	-123456789.11111
8 12345677.12345676 0.12345676	12345677
5 1234.1434 1234.2344	-.041
9 123478.667 123478.667	0

四行样例分别对应四种容易被忽略的输入规则：小数位要补齐、小数抵消后要收尾、整数部分为 0 时省略 0、全零只输出一个 `0`。这一题代码比较长，也比较容易出错，需要把每一步想清楚再写。

T9：样例

例子输入

```
5
10 .12345 123456789.23456
8 12345677.12345676 0.12345676
3 1112111.111111111111111 -111111.1111111
5 1234.1434 1234.2344
9 123478.667 123478.667
```

例子输出

```
-123456789.11111
12345677
2000222.222222211111111
-.041
0
```

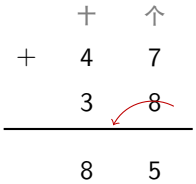
k 进制小数减法：五组样例（注意 -.041 与 0 这两种输出）

T9：已有类型的存储范围

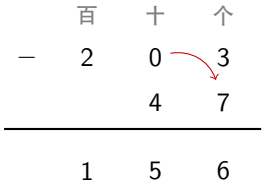
类型	字节	能表示的范围 / 精度	本题（≤ 100 位）
`int`	4	$\pm 2.1 \times 10^9$（约 10 位十进制）	不够
`long long`	8	$\pm 9.2 \times 10^{18}$（约 19 位）	不够
`double`	8	约 $15 \sim 17$ 位**有效数字**	不够，且十进制小数有误差

一是位数：100 位远超 `long long` 的 19 位。二是精度：`double` 只能保住前 16 位，而且它用二进制存小数，0.1 这种十进制小数本身就存不精确。所以只能用一个数组存每一位，自己处理进位与借位——这就是”高精度”。

T9：整数——加法的进位与减法的借位



$47 + 38 = 85$：**个位写 5、进 1**



$203 - 47 = 156$：**借 1 当 10**

```
r[i] = x[i] + y[i];
for (int i = 0; i < L-1; ++i) {
    r[i+1] += r[i] / bit;
    r[i]   %= bit;
}
```

```
r[i] = x[i] - y[i];
for (int i = 0; i < L-1; ++i)
    if (r[i] < 0) {
        r[i] += bit;  r[i+1] -= 1;
    }
```

两段循环都是从低位扫一遍：加法把进位加到下一位，减法"不够就 $+bit$、下一位减 1"。十位是 0 时的连锁借位不用特殊处理，扫描过程会自然完成。数组按"下标 0 是个位"存放，进位与借位都只是往右挪一格。

T9：处理小数

前面的加减都假设是整数，可本题有小数，而且两个数的小数位数可能不同：直接逐位相减会错位，因此需要将小数点对齐，需要一个比较好写的对齐做法。

办法：定点

把小数部分统一补 0 到 150 位，并把小数点固定在数组的同一个下标 $P = 150$ 上。这样两个数的小数点天然对齐，可以直接沿用前面的整数加减代码。

下标	$\cdots$	152	151	150	149	148	$\cdots$	0
含义	$\cdots$	百位	十位	个位	十分位	百分位	$\cdots$	第 150 位小数

关于补齐和数组

数的总长 ≤ 100，小数部分最多约 97 位，补到 150 位绰绰有余；这样补齐之后，最终数的位数最多可能是 250 位左右，再加上进位的一些影响，建议保存一个数的数组开大一些。

T9：读入处理

读入时先记下符号，再把字符串按小数点劈成整数部分（ip）与小数部分 (fp)。然后按下标放置：整数部分的个位落在 P，十位落在 $P+1$，依次往左；小数部分的十分位落在 $P-1$，百分位落在 $P-2$，依次往右。没有写到的位置保持 0，正好等价于补零，所以**不必真的手动添加** 0。

```
1 for (int i = 0; i < (int)ip.size(); ++i)
2     d[P + (int)ip.size() - 1 - i] = val(ip[i]);
3 for (int i = 0; i < (int)fp.size(); ++i)
4     d[P - 1 - i] = val(fp[i]);
```

其中 `val` 把字符转成数值：数字直接减`'0'`，字母减`'A'` 再加 10。整数部分为空（如`.1234`）或没有小数点（如 `1234`）都能被这两行正确处理。

T9：把减法化成加法

真正要处理的只有符号：记 a 的符号为 f_a、b 的符号为 f_b。同号时做绝对值减法，先比大小；异号时做绝对值加法，符号取 f_a。

情形	运算	结果符号
同号且 $\lvert a\rvert \geq \lvert b\rvert$	$\lvert a\rvert - \lvert b\rvert$	f_a
同号且 $\lvert a\rvert < \lvert b\rvert$	$\lvert b\rvert - \lvert a\rvert$	$\neg f_a$
异号	$\lvert a\rvert + \lvert b\rvert$	f_a

比较大小

要从高位到低位找第一个不同的位置，谁大谁就大；全部相同才是相等。

T9：输出规则

输出看起来琐碎，其实只有五条：结果为 0 就直接输出一个 `0`；有负号先输出负号；找到最高非零位与最低非零位；整数部分为 0 时以小数点开头；只输出到最低非零位，末尾的 0 自然就被丢掉了。

数值	正确输出	常见错误
五进制的 0.041 取负	`-.041`	`-0.041`、`-.41`
整数	`12345677`	`12345677.0`
二进制 110010	`110010`	`11001`（末位 0 被吞）
零	`0`	空行、`-0`

两个容易出错的事情：整数部分输出到下标 P（个位）就停止，否则小数位的的 0 会被当成整数部分的前导 0 删除；纯小数要先补 $P-1$ 到最高非零位之间的 0，否则 `-.041` 会变成 `-.41`。

T9：代码（一）字符转换与读入

```
const int P = 150, L = 505;         // P: 小数点位置, L: 数组长度
int bit, A[L], B[L], R[L];
int val(char c) { return (c <= '9') ? c - '0' : c - 'A' + 10; }
char ch(int x)  { return (x < 10) ? char('0' + x) : char('A' + x - 10); }

int readNum(const string &s, int *d) {   // 返回符号位
    int sign = (s[0] == '-'), dot = -1;
    for (int i = sign; i < (int)s.size(); ++i)
        if (s[i] == '.') dot = i;
    string ip, fp;
    if (dot == -1) ip = s.substr(sign);
    else { ip = s.substr(sign, dot - sign); fp = s.substr(dot + 1); }
    for (int i = 0; i < (int)ip.size(); ++i)
        d[P + (int)ip.size() - 1 - i] = val(ip[i]);
    for (int i = 0; i < (int)fp.size() && i < P; ++i)
        d[P - 1 - i] = val(fp[i]);
    return sign;
}
```

sign 既当符号（$0/1$），又当扫描起点（为 1 时正好跳过负号）；未写入的下标保持 0，等价于补零。

T9：代码（二）比较、加法与减法

```
int cmpAbs(int *x, int *y) {          // 返回 1 / 0 / -1
    for (int i = L - 1; i >= 0; --i)
        if (x[i] != y[i]) return x[i] > y[i] ? 1 : -1;
    return 0;
}
void add(int *x, int *y, int *r) {    // 绝对值相加
    for (int i = 0; i < L; ++i) r[i] = x[i] + y[i];
    for (int i = 0; i < L - 1; ++i) {
        r[i + 1] += r[i] / bit;       // 进位
        r[i]     %= bit;
    }
}
void sub(int *x, int *y, int *r) {    // 要求 x >= y
    for (int i = 0; i < L; ++i) r[i] = x[i] - y[i];
    for (int i = 0; i < L - 1; ++i)
        if (r[i] < 0) { r[i] += bit; r[i + 1] -= 1; }   // 借位
}
```

比较必须从高位往低位；两段循环都是低位到高位扫一遍，连锁进位与借位自然完成。

T9：代码（三）主函数

```
int main() {
    int T; cin >> T;
    while (T--) {
        string sa, sb;
        cin >> bit >> sa >> sb;          // 先读进制
        memset(A, 0, sizeof A);          // 多组数据必须清空
        memset(B, 0, sizeof B);
        memset(R, 0, sizeof R);
        int fa = readNum(sa, A), fb = readNum(sb, B), fs;
        if (fa == fb) {                  // 同号：绝对值相减
            fs = fa;
            if (cmpAbs(A, B) >= 0) sub(A, B, R);
            else { sub(B, A, R); fs ^= 1; }   // 大小反了，符号取反
        } else {                         // 异号：绝对值相加
            add(A, B, R);
            fs = fa;
        }
```

异号时符号取 fa（被减数的符号）：$-5-3=-8$、$5-(-3)=8$ 都符合；同号且被减数较小时要把符号取反。

T9：代码（四）输出

```
        bool zero = true;
        for (int i = 0; i < L; ++i)
            if (R[i]) { zero = false; break; }
        if (zero) { cout << 0 << '\n'; continue; }
        int h = L - 1, o = 0;
        while (h > 0 && R[h] == 0) --h;       // 最高非零位
        while (o < L - 1 && R[o] == 0) ++o;   // 最低非零位
        if (fs) cout << '-';
        if (h < P) {                           // 纯小数
            cout << '.';
            for (int i = P - 1; i > h; --i) cout << '0';
            for (int i = h; i >= o; --i) cout << ch(R[i]);
        } else {
            for (int i = h; i >= P; --i) cout << ch(R[i]);
            if (o < P) {                       // 还有小数部分
                cout << '.';
                for (int i = P - 1; i >= o; --i) cout << ch(R[i]);
            }
        }
        cout << '\n';
```

整数部分一直输出到下标 P，保证末位的 0 不被吞掉；纯小数分支先补中间的 0。

数据类型问题

需要记住 int 范围是 2.1×10^9，long long 是 9.2×10^{18}。
浮点数 double,float 看似能够保存很大的数，可是它们分别占用空间是 8 字节和 4 字节，所以能够存储的数的数量最多分别和 long long ，int 相同。因此浮点数后面的位的值可能就不准确了。浮点数尽量使用 double，因为 float 的误差会很大，常常导致意想不到的错误。由于误差比较浮点数是否相等，一般需要定义一个比较小的数 eps，abs(x-y)<eps 判断相同。

注意，一些运算的中间过程可能溢出：

```
int a = 100000, b = 100000;
long long c = a * b;        // 错：右边先按 int 算完再赋值
long long d = 1LL * a * b;  // 对：先转成 long long
//----------------------
int a,b,c<=10^9
cout<<a+b+c<<'\n';
1e17+1==1e17
```

数组：开在哪里、开多大、什么时候清

数组开在函数里是放在栈上的，因此对于本机，如果你在函数（包括 main）里开太大的数组，就算你的代码没有问题，也很容易导致运行时错误。开在 main 不占用栈的空间，而且默认全 0，开多少（只要不超过题目写的内存限制就可以）

```
1 const int MAXN = 1005;
2 int cnt[MAXN][MAXN];      // 全局：4MB，安全
3 int main() {
4     int a[1000000];       // 局部：4MB，危险
5 }
```

下标用到 n 就开 $n+5$；
不要用变量当数组长度（int a[n] 在某些编译器上可能会出错），
需要变量作为长度，或者不得不在函数里使用数组，就用 vector<int> a(n)。
数组和变量记得及时清空，尤其是涉及多组数据的情况。

另外，当在多组数据上测试错误的时候，不妨将每组数据拆开来，分别输入你的程序，如果分别输入的结果正确，那么你就需要检查是否完整地清空了所有可能用到的数组。

函数：返回值、值传递、数组传参

有返回类型的函数必须写 `return`，否则是未定义行为——本地也许能够编译通过，但是在编程网格上很可能会运行时错误或者结果不一致。

`void f(int x)` 修改形参不影响实参；要改实参就用引用 `void f(int &x)`，或者干脆让它操作全局数组 `void f(int *a)`。数组作参数时传的是首地址，
函数里的 `sizeof(a)` 不再是整个数组，所以必须额外传一个长度 `n`。

输入

输入基本是三种形态。第一种是" 先给个数、再给具体数据"，用 cin >> n 加一个循环即可。第二种是" 多组数据，读到文件末尾"，写成 while (cin >> n)，因为读到结尾，读入失败时 cin 会变成 false 自动跳出。第三种是" 一行一个字符串、其中可能含空格"，这时必须用 getline 读整行。

```
int n;
while (cin >> n) {            // 多组数据
    for (int i = 1; i <= n; ++i) cin >> a[i];
}
getline(cin, s);              // 整行，含空格
cin >> a >> c >> b;           // 用非空格字符分隔，如 12,34
while(getline(cin,s)) //每次读入一整行，直到文件末尾
```

输出

输出比较简单，题面怎么写就怎么输出，多组数据可能要输出多行，不要漏掉换行。一般行末多余的空格或者文末回车不会被判错，但是**一切以题目要求为准**。
当题目需要输出逗号时，切记要输出半角逗号。
像 cout<<', '; 可能会输出一些莫名奇妙的数字，因为全角的逗号其实占用两个字符。
需要注意的是浮点数的输出：

```
1 printf("%.3lf\n",x);//ok 保留三位小数 double 是 lf float 是 f
2 cout<<fixed<<setprecision(3)<<x<<endl;// ok 保留三位小数
```

额外提醒

cin>> 遇到空白或换行就停，但**不会取走**那个换行或空格；紧接着的 getline 一开始就读到换行，于是返回空串。表现是" 第一次读到空行，之后所有数据整体错位"。

```
int n, m; string st;
cin >> n >> m;
getchar();              // 抵消行末的换行 (也可 cin.ignore())
getline(cin, st);       // 现在才读到第一行文字
```

所以 cin >> T 之后要先把换行抵消。

排序与比较函数

`sort` 的区间是左闭右开，排 $a[1..n]$ 要写 `sort(a+1, a+n+1)`，右端少写一就漏掉最后一个元素；从大到小可以使用用 `greater<int>()`，对 `long long` 数组要写 `greater<long long>()`，当然也可以自己手写 `cmp`。

自定义规则时，`cmp` 必须满足"严格弱序"：自己不在自己前面，
也就是 `cmp(x,x)` 要为假。写成 `return a >= b` 就会破坏这一点，否则很容易造成**死循环**等问题。

`sort` **不保证稳定**，相等元素的先后由实现决定。所以凡是"相同则按输入顺序排"的题目，都要把"输入顺序"写在 `cmp` 里。如果你懒得写，可以使用 `stable_sort`，与 `sort` 的复杂度相同，但是能够保证比较结果相同时，输入顺序在前的排在前面。

复杂度：先估再写

数据规模	通常可以承受的复杂度	本套题中的例子
$n \le 2000$	$O(n^2 \log n)$	T6（$n \le 1000$）
$n \le 5000$	$O(n^2)$（常数不大）	——
$n \le 10^5$	$O(n\sqrt{n})$ 或 $O(n \log n)$	T4
$n \le 10^6$	$O(n \log n)$（常数一般即可）	T8（$N \le 10^6$，$O(n)$）
$n \le 10^7$	$O(n)$，常数比小	——

估计数量级

评测机每秒约 $10^8 \sim 2 \times 10^8$ 次简单运算，只要算出来低于这个范围就基本能够通过，考试试题不会为难复杂度正确的做法。10^6 个元素的排序约 0.1 秒；5000^2 的二重循环只要内层不是特别复杂也能过通过。

复杂度看似正确？

最常见的是一些操作使得复杂度其实增加了：典型的是 `strlen(s)` 求一个 char 数组的长度，其复杂度为 $O(n)$。
于是 `for (int i=0;i<strlen(s);++i)` 是 $O(n^2)$，正确的做法应该先用 `int len=strlen(s);` 保存长度。

还有两类容易被忽视：函数参数按值传递 `vector` 如 `void f(vector<int> a)`，这样每次调用都复制一遍；正确的做法是 `void f(vector<int> &a)`。
输入输出问题：用 `cin` 读入太多整数，输入 10^6 级别的时候会比较慢；用 `endl` 进行换行，这样换行每行都刷新缓冲。改成`'\n'` 会使得输出变快。`std::ios::sync_with_stdio(false);` 能够使得 `cin` 变快。(但是代价是什么呢?)

最后可能是循环：`while (x > 0)` 里忘了 `x /= 10`，或者循环变量在循环体里被改掉。

代码风格与一键格式化

如何写得清晰

变量名最好要说清它代表什么：`cnt`、`sum`、`age` 一眼能懂，像 `a1`、`b2` 很容易搞错。
缩进需要统一，否则容易弄错括号的层级。
一个函数只做一件事；数组的全局上界写成 `const int MAXN = 1005`，而不是 `a[1005],b[1005],c[1005]`，以防打错，循环的上界也建议写成全局的常量。
当一道题目比较复杂，涉及到函数或者非常多分支的时候，强烈建议写注释。

VS Code：Shift+Alt+F 一键格式化

装了 C/C++ 扩展之后，Shift+Alt+F 会按 clang-format 重排整个文件，右键菜单里的”格式化文档”是同一件事。想在保存时自动格式化，就打开设置里的 `editor.formatOnSave`。

Dev-C++：AStyle 菜单

菜单 AStyle 里有”格式化当前文件”，默认快捷键 Ctrl+Shift+A；

打开编译警告

何为编译警告

它能帮你在编译信息直接指出一些错误。如：变量未初始化就拿来用、函数少了 `return`、有符号与无符号混着比较、变量定义了却没用上，以及在 `if` 里把 `==` 写成 `=`。再配上 `-O2`，未定义行为会暴露得更明显，因为优化之后”碰巧能跑”的情况就少了。

VS Code

如果用 `tasks.json` 里的编译任务，就把`"-Wall"` 与`"-Wextra"` 加进 `args` 数组；也可以直接在终端敲 `g++ -Wall -Wextra -O2 a.cpp -o a`，

Dev-C++

打开工具菜单里的编译选项，在设置标签页勾上”在调用编译器时加入以下命令”，在后面的框里填 `-Wall -Wextra`。

警告不是错误，程序照样能生成。而且不是所有的错误都能够提前检查出来。

考前提醒

首先可以练习如何输入，否则遇到了一道会做的题，但是却不会输入那就很打击信心。

然后排序几乎是必考，建议熟练掌握；个人推荐直接使用 `sort`，不需要自己写。

考前提前一点到，可以配置一下自己喜欢的 IDE。当然也可以写一些模板。

祝大家考试顺利！